

Bookstore API Project Report

Project Title: RESTful Bookstore API

Author: [Your Name]

Date: [Submission Date]

1. Introduction

The Bookstore API is a backend application built using Java and Spring Boot for managing books and authors. It provides a RESTful interface that allows users to perform CRUD operations (Create, Read, Update, Delete) on books and authors. The API supports searching, pagination, sorting, and filtering, making it suitable for integration with frontend applications, mobile apps, or other services.

The project demonstrates the implementation of modern Spring Boot best practices, JPA for database interaction, and Swagger for API documentation. It uses the in-memory H2 database for development and testing, which is automatically created and populated as users interact with the API through REST calls (e.g., adding authors and books).

2. Objective

- Build a RESTful backend service for a bookstore.
 - Manage books and authors with full CRUD functionality.
 - Enable filtering, pagination, and sorting of books.
 - Provide documented API using Swagger UI.
 - Prepare the project for integration with client applications or Postman testing.
-

3. Tools & Technologies Used

Tool / Technology	Purpose
Java 17	Programming language
Spring Boot 4.0	Application framework for REST APIs
Spring Data JPA	ORM framework for database interaction
H2 Database	In-memory database for testing and development; tables are created dynamically as API calls are made
Maven	Build and dependency management
Postman	API testing and verification

Tool / Technology	Purpose
Swagger / OpenAPI	Automatic API documentation
VS Code	IDE for development

4. Project Structure

```

ELEVATE_Labs_Projects/
├── BookstoreAPI/
│   ├── src/
│   │   ├── main/
│   │   │   ├── java/com/example/bookstore/
│   │   │   │   ├── controller/
│   │   │   │   │   ├── BookController.java
│   │   │   │   │   └── AuthorController.java
│   │   │   │   ├── entity/
│   │   │   │   │   ├── Book.java
│   │   │   │   │   └── Author.java
│   │   │   │   ├── repository/
│   │   │   │   │   ├── BookRepository.java
│   │   │   │   │   └── AuthorRepository.java
│   │   │   │   ├── service/
│   │   │   │   │   ├── BookService.java
│   │   │   │   │   └── AuthorService.java
│   │   │   │   └── BookstoreApiApplication.java
│   │   │   └── resources/
│   │   │       ├── application.properties
│   │   │       ├── static/
│   │   │       └── templates/
│   │   └── test/
│   ├── pom.xml
│   ├── .gitignore
│   └── README.md

```

Explanation of Key Components:

- **Entities:** `Book` and `Author` represent database tables with relationships.
- **Repositories:** Provide database access using JPA.
- **Services:** Encapsulate business logic.
- **Controllers:** Handle HTTP requests for CRUD operations, filtering, pagination, and sorting.
- **Swagger UI:** Auto-generated API documentation.

5. Features Implemented

1. **CRUD Operations:** Create, Read, Update, Delete books and authors.
 2. **Filtering:** Search books by title and authors by name.
 3. **Pagination & Sorting:** Efficient retrieval of large datasets.
 4. **Relationships:** Books linked to authors using foreign keys.
 5. **API Documentation:** Swagger UI at `/swagger-ui/index.html`.
 6. **Testing:** Postman collection for testing all endpoints.
-

6. Application Workflow

1. **Run Application:** `BookstoreApiApplication` is the entry point. Spring Boot runs on `http://localhost:8080`.
2. **Interact with API:**
3. Add authors: `POST /api/authors`
4. Add books: `POST /api/books` with `authorId`
5. Retrieve: `GET /api/books` and `GET /api/authors`
6. Search, paginate, and sort using query parameters.
7. **View Documentation:** Swagger UI: `http://localhost:8080/swagger-ui/index.html`

Note: The H2 database is in-memory and automatically creates the necessary tables when you make REST calls. Data exists only during runtime unless explicitly saved elsewhere.

7. Deliverables

- Source code of the project.
 - Maven `pom.xml` with all dependencies.
 - Postman collection for all endpoints.
 - Swagger UI link for API documentation: <http://localhost:8080/swagger-ui/index.html>
 - H2 database tables created dynamically through API calls.
-

8. Conclusion

The Bookstore API demonstrates a full backend implementation using Java and Spring Boot, including relational database mapping, RESTful design, filtering, pagination, sorting, and API documentation. The application is modular, maintainable, and ready for integration with frontend or other applications. Future improvements could include authentication/authorization, frontend integration, and persistent production databases.